

CoScad Assembly Instruction Generator — Design

Sep 25, 2026 · @Jasen Qin

Motivation

Hand-assembling a 2020 T-slot frame from a CoScad model produced avoidable rework, and every mistake was an ordering or orientation mistake, not a design mistake. The model already knows what touches what; it should also tell you the order to build it.

Observed failures (Sep 2026, PETG panel enclosure build):

- Screws removed and re-inserted because a later part needed a slot the earlier bracket had already blocked.
- T-nuts sliding out of position in vertical rails while trying to hold two joints at once. Fix that worked: mark the rail, lay the rail being screwed into flat on the bench.
- Over-tightening printed T-slot lips (permanent deformation) with an IXO 7 that stalls at 3–5.5 Nm; printed lips want roughly 0.5–1 Nm.
- Bracket slot geometry (angled oval slots) making the driver sit off-axis and the bracket walk during tightening.

Each of these is a property of the assembly graph plus the parts' metadata, so a generator can predict and avoid them.

Goals and non-goals

The generator turns a `.assemble` file into a numbered build sequence that a person can follow without backtracking, with one render per step and a torque per fastener.

Goals:

- Zero-backtrack order: no step requires undoing an earlier step (no removed screws, no blocked slots).
- Nut-first discipline: every T-nut is inserted before the slot end it needs is consumed.
- Bench-stable steps: each step names which face is down and the partial assembly rests on it without support.
- Access-checked: the driver axis of every fastener is clear at the step it is tightened.

- Material-aware torque: aluminium vs printed rail decides the torque callout.
- Reuses existing machinery: the attachment graph, bounding boxes, and the planned clearance/interference checker.

Non-goals (v1):

- Full motion planning or swept-volume collision for inserting parts; bounding-box access checks are enough for frames.
- Deformable or press-fit parts, adhesives, wiring.
- Automatic fixture/jig generation (later, see Open questions).
- Multi-person or parallel build scheduling.

Inputs

The generator consumes only what the `.assemble` graph and part definitions already carry, plus a small amount of new per-part metadata.

Input	Source	Already exists?
Part instances with world transforms	<code>.assemble</code> placement semantics	Yes
Attachment edges (what is attached to what, via which faces)	<code>.assemble</code> attach/join system	Yes
Bounding boxes and joint geometry	CoScad bbox attachment system	Yes
Fastener instances (screw + nut + which slot/hole)	New: <code>fastener</code> construct in <code>.assemble</code>	No
Slot geometry per rail (which faces have slots, open ends)	New: part metadata on the 2020 constructor	No
Material class per part (<code>aluminium</code> , <code>printed</code> , <code>steel</code>)	New: part metadata, defaults from print/placement semantics	No
Driver clearance envelope per fastener type	New: fastener library table (head type, driver length, radius)	No
Mass per part	Volume × density from material class	Derivable

The only things the user writes by hand are the fastener declarations and, where the default is wrong, the material class. Everything else is derived.

Core model

An assembly is a graph whose nodes are parts and fasteners and whose edges are *requires-before* relations; a build plan is a linear extension of that graph that also satisfies orientation and access predicates at every prefix.

Node kinds:

- **Part:** a rigid body with transform, bbox, material class, mass, and zero or more *slots* (a slot = a face-aligned channel with two ends, each `open` or `blocked`).
- **Fastener:** a screw + optional nut, bound to a *host part* (the one it clamps to), a *clamped part* (bracket, panel), a *slot or hole* on the host, and a driver axis (unit vector in world space, pointing from the head outward).
- **Nut:** a drop-in or slide-in T-nut, bound to a slot; slide-in nuts additionally require an open slot end at insertion time.

Edges (all derived, none hand-written):

- `nut` → `fastener` : the fastener's nut must be present before the screw goes in.
- `nut(slide-in)` → any part that blocks the slot end it enters through.
- `clamped part` → `fastener` : a screw cannot go in before the part it clamps is placed.
- `host part` → `fastener` .
- `parent part` → `child part` from the attachment tree (a child is placed relative to its parent).

A **state** is the set of placed nodes plus a chosen *rest face* (the face of the partial assembly in contact with the bench). Predicates evaluated on a state:

- `stable(state)` : centre of mass projects inside the support polygon of the rest face, and the rest face is a real planar contact (not a screw head).
- `accessible(fastener, state)` : a cylinder along the driver axis (radius = driver clearance, length = driver length) does not intersect any placed part's bbox other than the clamped part.
- `nutCanEnter(nut, state)` : drop-in → slot face exposed; slide-in → at least one slot end still open.

```

flowchart LR
    R1[Rail A] --> N1[T-nut 1]
    R1 --> N2[T-nut 2]
    N1 --> F1[Screw 1]
    N2 --> F2[Screw 2]
    B1[Bracket] --> F1
    B1 --> F2
    R2[Rail B] --> F2
    F1 --> C1[Corner cap blocks slot end]

```

Reading: the corner cap can only go on after screw 1, and both nuts must be in Rail A before the bracket consumes the slot face.

Ordering algorithm

The planner is a best-first search over states, where each move places one node (or one *batch* of nuts into one rail) and may change the rest face; the hard predicates prune, and a cost function picks among the many valid linear extensions.

Why not a plain topological sort: a frame with 8 rails and 8 brackets has thousands of valid orders, and most of them flip the assembly repeatedly or leave it standing on a screw head. The sort gives correctness; the cost gives a build you would actually follow.

Search:

1. Start state: empty assembly, no rest face.
2. Moves from a state: every node whose requires-before predecessors are all placed. A nut move places *all* nuts for one rail at once (one "preload rail A" step). A rest-face change is a move with no placement (a "flip").
3. Apply hard predicates to the resulting state: `stable` , `accessible` for the fastener being tightened, `nutCanEnter` for nuts being inserted. Fail → prune.
4. Cost of a move (lower is better):
 - flip: 10
 - placing a part whose host rail is not horizontal on the bench: 6 (the "T-nut slides down" case)
 - placing a fastener whose driver axis is not within 30° of vertical-up: 3
 - placing a part that reduces the support polygon area: 2
 - placing a fastener that leaves a sibling fastener on the same bracket for later: 1 (finish a bracket before moving on)

5. Beam search (beam width ~50) with cost-so-far + a cheap admissible heuristic (remaining flips lower bound = number of distinct rest faces still needed). Frames are small; exact A* is also fine and can be the default until it isn't.
6. Post-pass: merge consecutive moves that share a rest face and a host part into one numbered step with sub-items.

$$\text{plan}^* = \arg\min_{\pi \in \text{LinExt}(G)} \sum_i c(s_{i-1}, \pi_i) \quad \text{s.t.} \quad \forall i: \text{stable}(s_i) \wedge \text{accessible}(\pi_i, s_{i-1})$$

The cost weights are a table in the planner config so they can be tuned per project (a wide flat base will tolerate flips; a tall narrow frame will not).

Constraints in detail

Each constraint maps to one of the failures in Motivation; the table records how it is detected and what the planner does about it.

Constraint	Failure it prevents	Detection	Planner response
Nut preloading	Nut inserted after the slot is blocked	<code>nutCanEnter</code> false for a required nut	Nut batch per rail is forced before the first fastener on that rail; slide-in nuts add a requires-before edge on the blocking part
Slot-end consumption	Corner cap / end bracket seals a slot still needed	A part's bbox covers a slot end that an unplaced slide-in nut needs	Requires-before edge: nut → blocking part
Driver access	Screw unreachable after neighbours placed	Driver cylinder vs placed bboxes	Fastener must be tightened before any part that would intersect its cylinder; if no order exists, report as a design error with the two parts named

Constraint	Failure it prevents	Detection	Planner response
Bench stability	Assembly tips, or rests on a screw head / nut	<code>stable(state)</code> ; rest face must be a planar contact of ≥ 2 parts or one rail face	Flip move inserted; cost 10
Horizontal host	T-nut slides down a vertical rail while aligning	Host rail axis not parallel to bench plane	Cost 6; the render marks the host rail
Vertical driving	Off-axis driving, bracket walking	Driver axis vs bench normal	Cost 3
Torque limit	Printed lips deform	Host material class	Torque callout on the step; <code>printed</code> $\rightarrow$ 0.6 Nm default, <code>aluminium</code> $\rightarrow$ 3 Nm, both overridable per part
Screw overlap	Two screws on a bracket collide	Two fastener shank cylinders intersect	Design error, not a planning error: reported before planning starts

Design errors vs planning errors: overlap and unreachable-fastener cases are properties of the model, so the generator reports them and stops. Everything else is resolved by ordering.

Output

The generator emits a plan as data (a `.plan` file) and renders it to Markdown + images; the data form is what the docsite and future FPRLive views consume.

Plan data, per step:

- Step number, rest face (which part face is down), and whether this step is a flip.
- Parts placed, with their instance names from `.assemble`.
- Fasteners tightened: screw spec, nut type, host part, slot/face, torque (Nm) and a plain-language equivalent ("finger tight + $\frac{1}{4}$ turn" for printed).
- Warnings: e.g. "rail B is vertical in this step; hold the nut with a finger".

Rendered outputs:

1. **Numbered instruction list** (Markdown) — one heading per step, sub-bullets for parts and fasteners.
2. **Per-step render** (OpenSCAD → PNG, one file per step) — placed parts opaque, parts placed *this* step highlighted, nothing else shown; camera looks down onto the rest face with a slight tilt so the bench orientation is obvious.
3. **Preload sheet** — one line per rail: how many nuts, which slots, where along the rail (mm from the marked end), so the marker-on-rail trick is done once up front.
4. **BOM** — fasteners and nuts grouped by spec, with counts, so you can lay them out before starting.
5. **Design error report** — overlaps and unreachable fasteners, with the two part names and a render of the conflict.

Example step (Markdown as rendered):

```
### Step 4 – rail A flat, slot face up
Place: bracket-A-B-1 on rail A at 380 mm
Tighten: M5×8 button head into T-nut #3 (rail A, top slot) – 3 Nm
Tighten: M5×8 button head into T-nut #4 (rail A, top slot) – 3 Nm
Note: rail B is not yet attached; leave bracket-A-B-1's second screw for step 6.
```

Language surface

The `.assemble` language gains one construct, `fastener`, and two optional annotations, `material` and `slots`; the planner is invoked with a `plan` directive. Both `!glyph` and `!simple` modes get the same additions.

`!simple` form:

```
!simple
```

```

railA  = Extrusion2020 400 |> material Aluminium
railB  = Extrusion2020 300 |> material Printed
br1    = Corner90

assembly =
  railA
  |> attach railB (end 1) (face Top) (at 380)
  |> attach br1  (inner corner railA railB)
  |> fastener M5x8 ButtonHead (DropIn) br1.hole1 railA.slotTop
  |> fastener M5x8 ButtonHead (DropIn) br1.hole2 railB.slotFront

plan assembly { restFaces = Auto, weights = Default }
```

Semantics:

- `material X` sets the material class; `Printed` implies the low torque default and a lip-load warning if a slide-in nut is used.
- `Extrusion2020 n` carries its slot metadata (four faces, both ends open) so nothing extra is written for standard rails. Custom rails declare `slots [Top, Front] ends [Open, Blocked]`.
- `fastener spec head nut clampedRef hostRef` binds a screw to a hole on the clamped part and a slot or hole on the host. The driver axis is derived from the hole's normal on the clamped part.
- `plan` runs the generator and writes `<name>.plan`, the Markdown, and the step renders next to the source file. `restFaces = [railA.Bottom, ...]` restricts allowed rest faces; `weights` overrides the cost table.

The `!glyph` form uses the same words as glyph aliases; `fastener` gets no glyph of its own — it is rare enough that a word is clearer.

Implementation plan

Four milestones, each usable on its own; the first two already pay for themselves on the enclosure build.

Milestone	Delivers	Depends on
M1 — Fastener model	<code>fastener</code> / <code>material</code> / <code>slots</code> in the <code>.assemble</code> grammar (both modes), fastener library table, derived requires-before graph, screw-overlap and preload-sheet output	Existing attach system
M2 — Ordered plan, no geometry	Topological sort with nut-first and slot-end edges, Markdown step list, BOM, torque callouts. No access or stability checks yet	M1
M3 — Geometry predicates	<code>accessible</code> via driver cylinders vs bboxes; <code>stable</code> via CoM and support polygon; flip moves; cost function and beam/A* search; design-error report	M2, clearance/interference checker
M4 — Renders	Per-step OpenSCAD scenes with highlight, camera on rest face, PNG export; hook into the docsite	M3

Testing: the enclosure frame is the first fixture. Hand-write the plan you wish you had followed, then check the generator's plan against it — differences are either a missing constraint or a weight to tune. Add a tall narrow frame and a printed-rail frame as the second and third fixtures.

- ☐ M1 — grammar + graph + preload sheet
- ☐ M2 — ordered Markdown plan
- ☐ M3 — access, stability, cost search
- ☐ M4 — step renders

Open questions

- ☐ Sub-assemblies: should the planner discover them (a bracket + two rails built flat, then lifted onto the frame) or should `.assemble` mark them explicitly? Discovery is a cut of the graph that stays stable on its own; explicit is simpler and matches the existing

recursive assemble decomposition.

- ☐ Torque numbers for printed lips: 0.6 Nm is a guess. Measure with a torque driver on a printed offcut and record per lip thickness and material (PETG vs PLA).
- ☐ Access check fidelity: bbox cylinders will false-positive on brackets whose real shape leaves the driver clear. Is a hull or convex decomposition of the part available cheaply from the existing geometry, or is a per-part `driverClear` override enough?
- ☐ Hand-holding steps: some orders are only possible with a third hand. Should the planner emit "clamp rail B to the bench" as a step, and does that need a fixture model?
- ☐ Rest faces on the enclosure: panels are fragile once fitted. Should the panel-fitting phase be a separate plan with `restFaces` restricted to unpanelled faces?
- ☐ Live view: the `.plan` data is a natural MVU model for a step-through view in the docsite or on the router/display box. Worth doing before M4, or after?